

Knowledge Graph Studio

Complete Project Report

Code Explanation · Data Flow · Database Schema · Architecture

AI-Powered Research Knowledge Graph Builder

Project	Knowledge Graph Studio
Version	2.0 — AI Enhanced
Backend	Python 3.11 + Flask
Database	MongoDB Atlas + SQLite Fallback
AI Model	Groq llama-3.3-70b-versatile
NLP Fallback	spaCy en_core_web_sm
Frontend	HTML5 + CSS3 + vis-network
Date	April 05, 2026

Table of Contents

1.	Project Overview & Features	3
2.	Technology Stack	4
3.	System Architecture	5
4.	Complete Program Flow	6
5.	File: app.py — Line by Line	7
6.	File: database.py — Line by Line	9
7.	File: ai_extractor.py — Line by Line	11
8.	File: nl_query.py — Line by Line	13
9.	File: extractor.py — Line by Line	14
10.	File: query_engine.py — Line by Line	15
11.	File: owl_parser.py — Line by Line	17
12.	File: processor.py — Line by Line	18
13.	Database Schema — How Data is Stored	19
14.	REST API Reference	21
15.	Frontend Architecture	22
16.	Interview Q&A;	23

1. Project Overview & Features

Knowledge Graph Studio is a full-stack web application that transforms research papers (PDFs) into interactive, queryable knowledge graphs. It uses AI (Groq LLM) to extract entities and relationships, stores them in MongoDB Atlas, and visualizes them as a force-directed graph in the browser.

What is a Knowledge Graph?

A knowledge graph is a network where nodes represent real-world entities (people, organizations, concepts) and edges represent relationships between them. Example:

Albert Einstein → DEVELOPED → Theory of Relativity → INFLUENCED → Modern Physics

Key Features

Feature	Description
■ AI PDF Extraction	Upload research papers — Groq LLM extracts entities and relationships automatically
■ Multi-PDF Fusion	Upload multiple papers at once — all graphs merge into one unified knowledge graph
■ Natural Language Query	Ask plain English questions — AI answers and highlights relevant nodes
■ Cypher-like Queries	MATCH, PATH, NEIGHBORS, STATS — query the graph like Neo4j
■ OWL/RDF/XML Import	Import ontologies from OWL files directly into the graph
■ MongoDB Atlas	Cloud persistence — data accessible from anywhere
■ SQLite Fallback	Works offline automatically if MongoDB is unavailable
■ Dark/Light Theme	Toggle between dark and light UI modes

2. Technology Stack

Backend

Technology	Version	Purpose
Python	3.11	Core runtime — 3.14 has TLS incompatibility with MongoDB Atlas
Flask	3.x	Web framework — handles all HTTP routes and API endpoints
PyMongo	4.x	MongoDB driver — connects to Atlas cloud database
spaCy	3.8	NLP — Named Entity Recognition + dependency parsing (fallback)
pdfplumber	0.11	PDF text extraction — handles multi-page documents
Groq SDK	latest	LLM API client — calls llama-3.3-70b for AI extraction
owlready2	0.50	OWL ontology processing and RDF/XML generation
networkx	3.x	Graph algorithms — BFS shortest path calculation
python-dotenv	1.x	Loads .env file — keeps API keys out of code
Werkzeug	3.x	Secure file upload handling — prevents path traversal

Frontend & Database

Technology	Purpose
HTML5 / CSS3 / Vanilla JS	Single-page application — no build tools needed
vis-network (CDN)	Interactive graph visualization with physics simulation
MongoDB Atlas (Free M0)	Primary cloud database — stores all nodes and edges
SQLite (graph.db)	Local fallback database — works without internet
Groq API (Free tier)	llama-3.3-70b — PDF extraction + NL query answering

3. System Architecture

The application uses a three-tier architecture: browser frontend, Python/Flask backend, and cloud database. The AI layer (Groq) enhances extraction and querying capabilities.

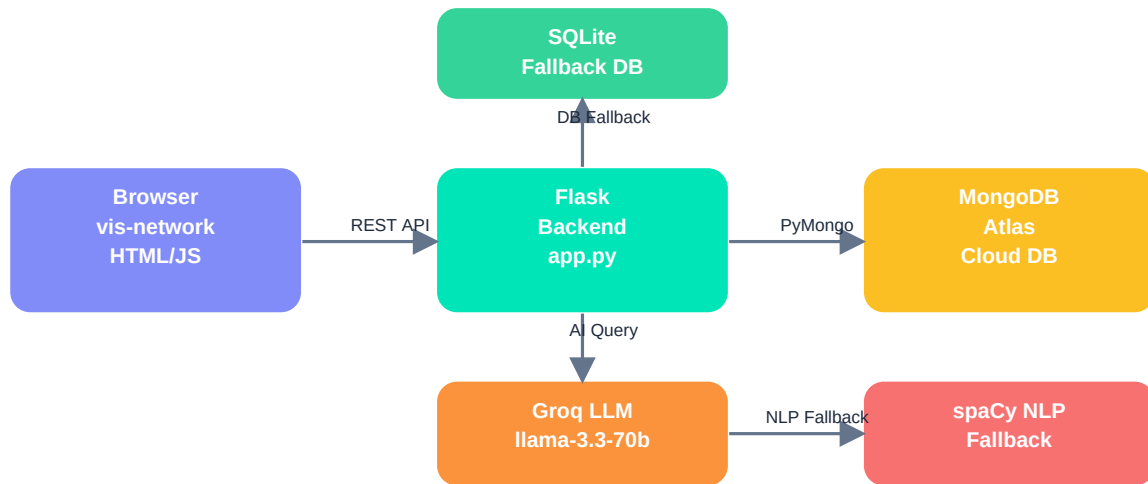


Figure 1: System Architecture — all components and their connections

Project File Structure

File	Role	Size
app.py	Flask web server — all REST API routes	~160 lines
database.py	MongoDB/SQLite abstraction layer	~180 lines
ai_extractor.py	LLM-powered PDF → knowledge graph	~100 lines
nl_query.py	Natural language question answering	~90 lines
extractor.py	spaCy NLP fallback extractor	~80 lines
query_engine.py	Cypher-like query language engine	~400 lines
owl_parser.py	OWL/RDF/XML → graph JSON parser	~100 lines
processor.py	PDF → OWL ontology (owlready2)	~150 lines
templates/index.html	Complete frontend SPA	~1100 lines
.env	API keys and DB credentials (not in git)	3 lines

4. Complete Program Flow

4.1 Application Startup

When you run **py -3.11 app.py**, this sequence happens:

```
py -3.11 app.py
```

```
■■■ load_dotenv()          # reads .env → MONGO_URI, GROQ_API_KEY into os.environ
■■■ init_db()              # tries MongoDB Atlas → creates indexes
■    ■■■ if fails → SQLite # creates graph.db with nodes+edges tables
■■■ migrate_json_if_needed() # imports old graph_db.json if it exists
■■■ Flask starts on :5000   # ready to accept browser requests
```

4.2 PDF Upload & AI Extraction Flow



Figure 2: PDF to Knowledge Graph — 6-step AI extraction pipeline

```
POST /upload
```

```
■■■ secure_filename() → save PDF to /uploads/
■■■ ai_extractor.extract_graph_ai(path)
■    ■■■ pdfplumber.open() → extract text from all pages
■    ■■■ split text into 3000-char chunks (max 10 chunks)
■    ■■■ for each chunk:
■        ■■■ Groq API call → llama-3.3-70b
■        ■■■ returns JSON: {entities:[...], relations:[...]}
■        ■■■ deduplicate + merge into all_entities, all_relations
■    [if Groq fails → fallback to spaCy NER + SVO parsing]
■■■ for each entity → insert_node() → MongoDB nodes collection
■■■ for each relation → insert_edge() → MongoDB edges collection
■■■ return {graph, stats, method} → browser renders graph
```

4.3 Natural Language Query Flow



Figure 3: Natural Language Query — question to highlighted graph nodes

POST /ask

■■■ load_db() → fetch all nodes+edges from MongoDB

■■■ _build_graph_summary() → compact text (max 150 nodes, 150 edges)

■■■ Groq API: question + graph summary → llama-3.3-70b

■■■ returns: {answer, relevant_node_ids, confidence}

■■■ validate node IDs exist in DB

■■■ return answer + IDs → frontend dims other nodes, zooms to relevant

4.4 Manual CRUD Flow

POST /node → validate label → next_node_id() → insert_node() → return graph

PUT /node/ → load_db() → update_node() → return graph

DELETE /node/ → delete_node() + delete all connected edges → return graph

POST /edge → check both nodes exist → next_edge_id() → insert_edge() → return graph

5. File: app.py — Line by Line

app.py is the entry point and web server. It defines all HTTP routes and connects every module together.

Lines 1-5: Imports

```
from flask import Flask, request, jsonify, render_template
from werkzeug.utils import secure_filename
from dotenv import load_dotenv
load_dotenv()
```

Explanation: Flask provides the web framework. `secure_filename()` sanitizes uploaded filenames to prevent path traversal attacks (e.g. `../etc/passwd`). `load_dotenv()` reads the `.env` file and puts `MONGO_URI` and `GROQ_API_KEY` into `os.environ` before anything else runs.

Lines 6-11: Import database functions

```
from database import (
    init_db, migrate_json_if_needed, load_db, db_to_vis,
    insert_node, update_node, delete_node, node_exists,
    insert_edge, delete_edge, edge_key_exists,
    next_node_id, next_edge_id, clear_all
)
```

Explanation: All database operations are imported from `database.py`. `app.py` never talks to MongoDB or SQLite directly — it only calls these functions. This is the abstraction layer pattern.

Lines 13-16: App setup

```
app = Flask(__name__)
UPLOAD_FOLDER = os.path.join(os.path.dirname(__file__), 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['MAX_CONTENT_LENGTH'] = 50 * 1024 * 1024
```

Explanation: Creates the Flask app. Sets upload folder to `uploads/` next to `app.py`. `makedirs` creates it if missing. `MAX_CONTENT_LENGTH` limits uploads to 50MB — prevents memory exhaustion attacks.

Lines 19-20: Startup

```
init_db()
migrate_json_if_needed()
```

Explanation: Runs once when app starts. `init_db()` connects to MongoDB and creates indexes (or initializes SQLite). `migrate_json_if_needed()` imports any old `graph_db.json` data if that file exists.

GET / → index()

```
@app.route('/')
def index():
    return render_template('index.html')
```

Explanation: When browser opens `localhost:5000`, Flask reads `templates/index.html` and sends it. This is the entire frontend — one HTML file with all CSS and JS inside.

GET /graph → get_graph()

```
@app.route('/graph', methods=['GET'])
def get_graph():
    return jsonify(db_to_vis())
```

Explanation: Called by the browser on page load and after every change. `db_to_vis()` fetches all nodes and edges from MongoDB and formats them for vis-network: `{nodes:[...], edges:[...]}`. The `?t=timestamp` query param prevents browser caching.

POST /node → add_node()

```
label = data.get('label', '').strip()
nid = next_node_id(label)      # e.g. 'n5_JohnSmith'
while node_exists(nid):       # ensure uniqueness
    nid += '_'
insert_node(nid, label, ntype, props)
return jsonify({'id': nid, 'graph': db_to_vis()})
```

Explanation: Creates a new node. `next_node_id()` generates `'n{count}_{label}'` — the count prefix ensures uniqueness, the label suffix makes it human-readable. The while loop handles rare collisions. Returns the full updated graph so the browser re-renders immediately.

DELETE /node/ → delete_node_route()

```
@app.route('/node/', methods=['DELETE'])
def delete_node_route(nid):
    if not node_exists(nid):
        return jsonify({'error': 'Node not found'}), 404
    delete_node(nid) # also deletes all connected edges
    return jsonify({'graph': db_to_vis()})
```

Explanation: Deletes a node AND all its edges. The 404 check prevents silent failures. `delete_node()` in `database.py` uses MongoDB's `$or` operator to delete edges where `from_id` OR `to_id` matches — keeps the graph consistent.

POST /ask → ask()

```
@app.route('/ask', methods=['POST'])
def ask():
    from nl_query import answer_question
    question = data.get('question', '').strip()
    result = answer_question(question, load_db())
    return jsonify(result)
```

Explanation: AI natural language query. Imports `nl_query` lazily (only when needed). Loads the full graph from MongoDB, passes it with the question to Groq LLM, returns the answer + relevant node IDs for highlighting.

POST /upload → upload()

```
file.save(path)
from ai_extractor import extract_graph_ai
result = extract_graph_ai(path)
for n in result['nodes']:
    if not node_exists(nid):
        insert_node(...)
for e in result['edges']:
    if not edge_key_exists(...):
        insert_edge(...)
```

Explanation: Saves the PDF, runs AI extraction, then inserts only NEW nodes/edges. The `node_exists()` and `edge_key_exists()` checks prevent duplicates when the same PDF is uploaded twice. Returns stats (nodes/edges added) and the full updated graph.

6. File: database.py — Line by Line

database.py is the data access layer. It abstracts MongoDB and SQLite behind a single API so the rest of the code never needs to know which database is active.

Module-level globals (singleton pattern)

```
_client = None
_db      = None
_use_mongo = False
```

Explanation: These three variables are set once when the app starts and reused for every request. This is the singleton pattern — creating one database connection and sharing it, rather than reconnecting on every request (which would be very slow).

get_db() — connection with fallback

```
def get_db():
    global _client, _db, _use_mongo
    if _db is not None:
        return _db          # already connected, reuse
    _client = MongoClient(MONGO_URI,
                          serverSelectionTimeoutMS=10000,
                          tlsInsecure=True) # bypass TLS cert validation
    _client.admin.command('ping') # test connection
    _db = _client[DB_NAME]
    _use_mongo = True
```

Explanation: `tlsInsecure=True` bypasses TLS certificate validation — needed because Python 3.11's OpenSSL is strict with MongoDB Atlas's TLS configuration. The `ping` command forces an actual connection test immediately so we know if MongoDB is reachable before any data operations.

Fallback to SQLite

```
except Exception as e:
    print(f'MongoDB failed, falling back to SQLite')
    _use_mongo = False
    _db = 'sqlite'
    return _db
```

Explanation: If MongoDB fails for ANY reason (network, wrong credentials, SSL error), the code silently falls back to SQLite. The string `'sqlite'` is stored in `_db` as a sentinel value. `_is_mongo()` checks `_use_mongo` flag to decide which path to take.

init_db() — create indexes

```
def init_db():
    if _is_mongo():
        _db.nodes.create_index('id', unique=True)
        _db.edges.create_index('id', unique=True)
        _db.edges.create_index([
            ('from_id', ASCENDING),
            ('to_id', ASCENDING),
            ('label', ASCENDING)
        ])
```

Explanation: Creates MongoDB indexes on startup. The unique index on nodes.id prevents duplicate nodes. The compound index on edges (from_id, to_id, label) makes edge_key_exists() fast and prevents duplicate relationships between the same pair of nodes.

insert_node() — upsert pattern

```
def insert_node(nid, label, ntype, properties):
    if _is_mongo():
        _db.nodes.update_one(
            {'id': nid},
            {'$set': {'id':nid,'label':label,'type':ntype,'properties':properties}},
            upsert=True
        )
```

Explanation: upsert=True means: UPDATE if a document with this id exists, INSERT if not. This prevents duplicates when the same PDF is uploaded twice — the second upload just updates existing nodes rather than creating duplicates.

delete_node() — cascade delete

```
def delete_node(nid):
    if _is_mongo():
        _db.nodes.delete_one({'id': nid})
        _db.edges.delete_many({
            '$or': [{'from_id': nid}, {'to_id': nid}]
        })
```

Explanation: The \$or operator deletes ALL edges where this node appears as either source (from_id) or target (to_id). This cascade delete keeps the graph consistent — no orphaned edges pointing to deleted nodes.

next_node_id() — ID generation

```
def next_node_id(label):
    count = _db.nodes.count_documents({})
    return 'n' + str(count+1) + '_' + re.sub(r'\W+', '', label)[:12]
```

Explanation: Generates IDs like 'n5_JohnSmith'. The number prefix (count+1) ensures uniqueness even if two nodes have the same label. The label suffix (first 12 alphanumeric chars) makes IDs human-readable for debugging. re.sub removes special characters.

load_db() — fetch all data

```
def load_db():
    if _is_mongo():
        nodes = {}
        for doc in _db.nodes.find({}, {'_id': 0}):
            nodes[doc['id']] = {'label': doc['label'], ...}
        edges = [{'id': d['id'], 'from': d['from_id'], ...}
                  for d in _db.edges.find({}, {'_id': 0})]
        return {'nodes': nodes, 'edges': edges}
```

Explanation: {'_id': 0} excludes MongoDB's internal _id field from results. Nodes are returned as a dict keyed by id for O(1) lookup. Edges are a list. This structure is used by the query engine, AI query, and visualization.

7. File: ai_extractor.py — Line by Line

ai_extractor.py is the AI brain of the project. It reads PDFs, splits them into chunks, sends each chunk to Groq's LLM, and collects structured entity/relationship data.

Constants

```
CHUNK_SIZE = 3000    # characters per chunk
MAX_CHUNKS = 10      # max chunks per PDF
```

Explanation: 3000 chars is roughly 500 words — enough context for the LLM to understand relationships without exceeding token limits. MAX_CHUNKS=10 means max 30,000 chars (about 10 pages) processed per PDF, controlling API usage and cost.

SYSTEM_PROMPT — instructing the LLM

```
SYSTEM_PROMPT = """You are a knowledge graph extraction expert.
Return ONLY valid JSON:
{
  "entities": [{"id": "snake_case", "label": "Name", "type": "Person|Org|..."}],
  "relations": [{"from": "id", "relation": "USES", "to": "id"}]
}"""
```

Explanation: The system prompt is the most important part. It tells the LLM exactly what format to return. temperature=0.1 makes output deterministic and factual (not creative). The strict JSON format requirement means we can reliably parse the response.

_chunk_text() — smart splitting

```
def _chunk_text(text, size=CHUNK_SIZE):
    sentences = re.split(r'(?<=[!?!])\s+', text)
    chunks, current = [], ''
    for s in sentences:
        if len(current) + len(s) > size and current:
            chunks.append(current.strip())
            current = s
        else:
            current += ' ' + s
```

Explanation: Splits on sentence boundaries (!?) rather than arbitrary character positions. This keeps sentences intact so the LLM gets coherent context. A sentence is never cut in the middle, which would confuse the LLM about entity relationships.

extract_graph_ai() — main function

```
def extract_graph_ai(pdf_path):
    if not GROQ_API_KEY:
        from extractor import extract_graph
        return extract_graph(pdf_path) # spaCy fallback
    text = _extract_text(pdf_path)
    chunks = _chunk_text(text)
    all_entities = {} # id -> node dict
    all_relations = []
    seen_edges = set() # prevent duplicates
```

Explanation: If no API key is configured, silently falls back to spaCy. `all_entities` is a dict keyed by ID — this automatically deduplicates entities across chunks (same entity mentioned in chunk 1 and chunk 5 only appears once). `seen_edges` is a set of (from,to,relation) tuples to prevent duplicate edges.

Processing each chunk

```
for i, chunk in enumerate(chunks):
    try:
        raw = _call_groq(chunk)
        data = _parse_llm_response(raw)
        for ent in data.get('entities', []):
            eid = _safe_id(ent.get('id') or ent.get('label', ''))
            if eid not in all_entities: # deduplicate
                all_entities[eid] = {...}
    except Exception as e:
        errors += 1
        continue # skip bad chunks, don't crash
```

Explanation: The try/except around each chunk means one bad LLM response doesn't crash the whole extraction. `errors` counter tracks failures. If ALL chunks fail, it falls back to spaCy. `_safe_id()` converts any text to valid snake_case ID.

Fallback logic

```
if errors == len(chunks):
    from extractor import extract_graph
    result = extract_graph(pdf_path)
    result['method'] = 'spacy_fallback'
    return result
```

Explanation: Only falls back to spaCy if EVERY chunk failed. If even one chunk succeeded, we use the AI results. This ensures maximum AI coverage while guaranteeing the user always gets some result.

8. File: `nl_query.py` — Line by Line

`nl_query.py` handles natural language questions about the graph. It sends the question + a compact graph summary to Groq LLM and returns a human-readable answer plus the IDs of relevant nodes.

`_build_graph_summary()` — token management

```
def _build_graph_summary(db):
    node_items = list(nodes.items())[:150] # max 150 nodes
    edge_items = edges[:150] # max 150 edges
    node_str = '\n'.join(
        f'{{"id": "{nid}", "label": "{n["label"]}", "type": "{n["type"]}"}}'
        for nid, n in node_items
    )
```

Explanation: Limits to 150 nodes and 150 edges to avoid exceeding the LLM's context window (token limit). The graph is serialized as compact JSON strings — no whitespace — to minimize token usage. Sending the full graph for large knowledge graphs would exceed limits and cost more.

`answer_question()` — main function

```
resp = client.chat.completions.create(
    model='llama-3.3-70b-versatile',
    messages=[
        {'role': 'system', 'content': SYSTEM_PROMPT},
        {'role': 'user', 'content': f'Question: {question}\n\nGraph: \n{graph_summary}'}
    ],
    temperature=0.1, max_tokens=1024
)
```

Explanation: The user message contains both the question AND the graph data. The LLM reads the graph, finds relevant nodes, and answers the question. `temperature=0.1` keeps answers factual and consistent.

Validating LLM response

```
valid_nodes = [nid for nid in result.get('relevant_node_ids', []) if nid in nodes]
valid_edges = [e['id'] for e in edges if e['id'] in result.get('relevant_edge_ids', [])]
```

Explanation: The LLM might hallucinate node IDs that don't exist. This validation filters out any IDs not present in the actual database. Only real, existing node IDs are returned to the frontend for highlighting.

9. File: extractor.py — Line by Line

extractor.py is the spaCy-based fallback extractor. It uses NLP (Natural Language Processing) to extract entities and relationships without needing an API key.

Loading spaCy model

```
try:
    nlp = spacy.load('en_core_web_lg') # best quality
except OSError:
    try:
        nlp = spacy.load('en_core_web_md') # medium
    except OSError:
        nlp = spacy.load('en_core_web_sm') # minimum
```

Explanation: Tries to load the largest model first (best accuracy), falls back to smaller ones. This runs at import time — the model loads once into memory and is reused for all PDFs. Loading takes 1-2 seconds but subsequent uses are fast.

NER type mapping

```
NER_TYPE_MAP = {
    'PERSON': 'Person', 'ORG': 'Organization',
    'GPE': 'Location', 'LOC': 'Location',
    'DATE': 'Date', 'EVENT': 'Event',
    'PRODUCT': 'Product', ...
}
```

Explanation: Maps spaCy's internal NER labels to our graph node types. GPE (Geopolitical Entity) maps to Location. WORK_OF_ART, LAW, LANGUAGE all map to Concept. This standardizes the type vocabulary across the whole graph.

SVO triple extraction

```
for token in sent:
    if token.pos_ != 'VERB': continue
    subjs = [c for c in token.children
              if c.dep_ in ('nsubj', 'nsubjpass')]
    objs = [c for c in token.children
            if c.dep_ in ('dobj', 'attr', 'pobj')]
    for s in subjs:
        for o in objs:
            add_edge(s.text, token.lemma_, o.text)
```

Explanation: Subject-Verb-Object extraction using dependency parsing. For each VERB token, finds its grammatical subject (nsubj) and object (dobj). Example: 'Einstein developed relativity' → subject=Einstein, verb=developed, object=relativity → edge: Einstein -[developed]→ relativity. token.lemma_ gives the base form (developed→develop).

Co-occurrence extraction

```
for sent in doc.sents:
    ents = [e.text for e in sent.ents]
    for i in range(len(ents) - 1):
        if ents[i] != ents[i+1]:
            add_edge(ents[i], 'related_to', ents[i+1])
```

Explanation: If two named entities appear in the same sentence, they're probably related. This creates 'related_to' edges between consecutive entities in each sentence. Less precise than SVO but catches relationships that don't have explicit verbs.

10. File: query_engine.py — Line by Line

query_engine.py implements a mini Cypher-like query language. It parses text queries and executes them against the in-memory graph data.

run_query() — router

```
def run_query(q, db):
    upper = q.upper()
    if upper.startswith('STATS'):    return _stats(db)
    if upper.startswith('NEIGHBORS'): return _neighbors(q, db)
    if upper.startswith('PATH'):     return _path(q, db)
    if upper.startswith('COUNT'):   return _count(q, db)
    if upper.startswith('MATCH'):    return _match(q, db)
```

Explanation: Simple keyword-based router. Checks the first word of the query and dispatches to the right handler. All handlers return the same structure: {columns, rows, graph, message} so the frontend can render them uniformly.

_match() — pattern parsing with regex

```
rel_pattern = re.search(
    r'\((\w*):?(\w*)\)\s*-\[ (\w*):?(\w*)\]->\s*(\w*):?(\w*)\)', q
)
```

Explanation: This regex parses (n:Person)-[r:WORKS_AT]->(m:Organization). The 6 capture groups are: node1_var, node1_type, rel_var, rel_type, node2_var, node2_type. If this pattern matches, it's a relationship query. If not, it falls through to node-only matching.

WHERE clause evaluation

```
m = re.match(r'(\w+)\.(\w+)\s+CONTAINS\s+"([^\"]*)"', clause)
if m:
    var, field, val = m.groups()
    return val.lower() in str(_get_field(row, var, field)).lower()
```

Explanation: Evaluates WHERE conditions like 'n.label CONTAINS "Einstein"'. Supports CONTAINS, STARTS WITH, = (equals), != (not equals), AND, OR. _get_field() extracts the right value from the result row based on variable name and field name.

_path() — BFS shortest path

```
adj = {}
for e in edges:
    adj.setdefault(e['from'], []).append((e['to'], e['label'], e['id']))
    adj.setdefault(e['to'], []).append((e['from'], e['label'], e['id']))
queue = deque([(src_id, [src_id], [])])
visited = {src_id}
```

Explanation: Builds an adjacency list treating edges as undirected (both directions). BFS (Breadth-First Search) guarantees the shortest path is found first. Each queue item carries (current_node, path_nodes_so_far, path_edges_so_far). The visited set prevents infinite loops in cyclic graphs.

`_stats()` — graph statistics

```
type_counts = {}
for n in nodes.values():
    t = n.get('type', 'Unknown')
    type_counts[t] = type_counts.get(t, 0) + 1
rel_counts = {}
for e in edges:
    rel_counts[e['label']] = rel_counts.get(e['label'], 0) + 1
```

Explanation: Counts nodes by type and edges by relationship label. Returns a table showing total nodes, total edges, breakdown by entity type, and breakdown by relationship type. Useful for understanding the composition of your knowledge graph.

11. File: owl_parser.py — Line by Line

owl_parser.py parses OWL/RDF/XML ontology files into graph JSON. It uses Python's built-in xml.etree — no external OWL library needed for parsing.

XML Namespaces

```
RDF = 'http://www.w3.org/1999/02/22-rdf-syntax-ns#'
RDFS = 'http://www.w3.org/2000/01/rdf-schema#'
OWL = 'http://www.w3.org/2002/07/owl#'
```

Explanation: OWL files use XML namespaces as prefixes for all tags. Every owl:Class tag is actually {http://www.w3.org/2002/07/owl#}Class in the XML. These constants are used to find elements: root.findall(f'{{{OWL}}}Class').

_local() — URI to name

```
def _local(uri):
    for sep in ['#', '/']:
        if sep in uri:
            return uri.rsplit(sep, 1)[-1]
    return uri
```

Explanation: Extracts the local name from a URI. 'http://ontology#Person' → 'Person'. 'http://ontology/concepts/BERT' → 'BERT'. rsplit with maxsplit=1 splits only on the last occurrence of the separator.

Parsing NamedIndividuals (nodes)

```
for el in root.findall(f'{{{OWL}}}NamedIndividual'):
    ind_id = _local(el.get(f'{{{RDF}}}about', ''))
    label_el = el.find(f'{{{RDFS}}}label')
    label = label_el.text if label_el else ind_id
    type_el = el.find(f'{{{RDF}}}type')
    etype = _local(type_el.get(f'{{{RDF}}}resource', ''))
```

Explanation: NamedIndividuals are the actual instances in an ontology — they become graph nodes. rdf:about gives the URI (converted to ID). rdfs:label gives the display name. rdf:type gives the class (entity type).

DOCTYPE stripping

```
clean = re.sub(r']*\[.*?\]>', '', text, flags=re.DOTALL)
root = ET.fromstring(clean)
```

Explanation: Python's xml.etree doesn't support DOCTYPE declarations. The regex strips them before parsing. re.DOTALL makes . match newlines so multi-line DOCTYPE blocks are removed correctly.

12. File: processor.py — Line by Line

processor.py converts PDFs into OWL ontology files using owlready2. It's the most complex file — it dynamically creates OWL classes and properties at runtime.

Fresh world per call

```
owlready2.default_world = owlready2.World()
unique_iri = f'http://research-kg.org/ontology/{uuid.uuid4().hex}#'
onto = owlready2.get_ontology(unique_iri)
```

Explanation: owlready2 caches ontology classes globally. If you process two PDFs, the second call would reuse classes from the first, causing metaclass conflicts. Creating a fresh World() and using a unique UUID-based IRI prevents this completely.

Dynamic class creation

```
for label, cls_name in NER_CLASS_MAP.items():
    cls = types.new_class(cls_name, (ResearchEntity,))
    cls.namespace = onto
    ner_classes[label] = cls
```

Explanation: types.new_class() creates Python classes dynamically at runtime. Each NER type (PERSON, ORG, etc.) becomes an OWL class that inherits from ResearchEntity. This is equivalent to writing 'class Person(ResearchEntity): pass' but done programmatically.

Dynamic property creation

```
for rel in {t[1] for t in triples}:
    prop = types.new_class(prop_name, (owlready2.ObjectProperty,))
    prop.domain = [ResearchEntity]
    prop.range = [ResearchEntity]
```

Explanation: Each unique verb extracted from the PDF becomes an OWL ObjectProperty. domain and range constrain what types of entities can be connected by this property. The result is a proper OWL ontology that can be opened in Protégé.

13. Database Schema — How Data is Stored

All graph data is stored in MongoDB Atlas (cloud) with SQLite as an automatic offline fallback. The schema is identical in both databases.

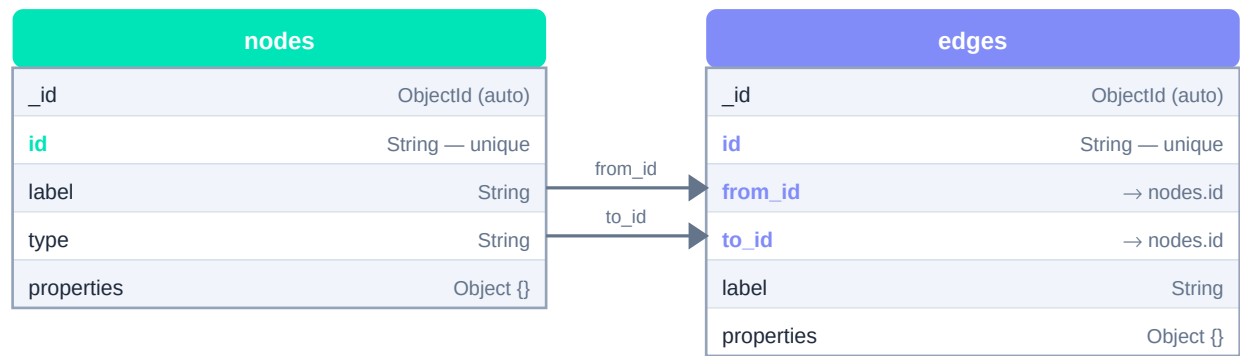


Figure 4: Database schema — nodes and edges collections with relationships

MongoDB: nodes collection

Field	Type	Example	Notes
_id	ObjectId	ObjectId('69c5...')	Auto-generated by MongoDB
id	String	john_smith	Unique — indexed. Used as graph node ID
label	String	John Smith	Display name shown on graph
type	String	Person	Entity type: Person Org Location Concept ...
properties	Object	{'born':1879}	Key-value metadata. Empty {} if none

MongoDB: edges collection

Field	Type	Example	Notes
_id	ObjectId	ObjectId('69c5...')	Auto-generated by MongoDB
id	String	e5	Unique edge ID. Format: e{count}
from_id	String	john_smith	References nodes.id (source node)
to_id	String	stanford_university	References nodes.id (target node)
label	String	AFFILIATED_WITH	Relationship type — uppercase snake_case
properties	Object	{'since':2020}	Key-value metadata. Empty {} if none

MongoDB Indexes

Collection	Index	Type	Purpose
nodes	id	Unique	Prevents duplicate nodes. Fast lookup by ID

edges	id	Unique	Prevents duplicate edge IDs
edges	(from_id, to_id, label)	Compound	Prevents duplicate relationships. Fast edge_key_exists()

SQLite Schema (Fallback)

```
CREATE TABLE nodes (  
  id          TEXT PRIMARY KEY,  -- unique node identifier  
  label       TEXT NOT NULL,     -- display name  
  type        TEXT DEFAULT 'Node', -- entity type  
  properties  TEXT DEFAULT '{}',  -- JSON string  
);  
  
CREATE TABLE edges (  
  id          TEXT PRIMARY KEY,  -- unique edge identifier  
  from_id     TEXT NOT NULL,     -- source node id  
  to_id       TEXT NOT NULL,     -- target node id  
  label       TEXT NOT NULL,     -- relationship type  
  properties  TEXT DEFAULT '{}',  -- JSON string  
);
```

How Data Flows: PDF → MongoDB

Step	What Happens	Where Stored
1. Upload	PDF saved to /uploads/ folder	Local filesystem
2. Extract	pdfplumber reads all pages	RAM (temporary)
3. Chunk	Text split into 3000-char pieces	RAM (temporary)
4. LLM call	Each chunk sent to Groq API	Groq servers (API call)
5. Parse	JSONs and relations extracted	RAM (temporary)
6. Deduplicate	Same entity from multiple chunks merged	RAM (temporary)
7. Insert	insert_node() with upsert=True	MongoDB nodes collection
8. Insert	insert_edge() if not exists	MongoDB edges collection
9. Return	display() fetches all data	Sent to browser as JSON

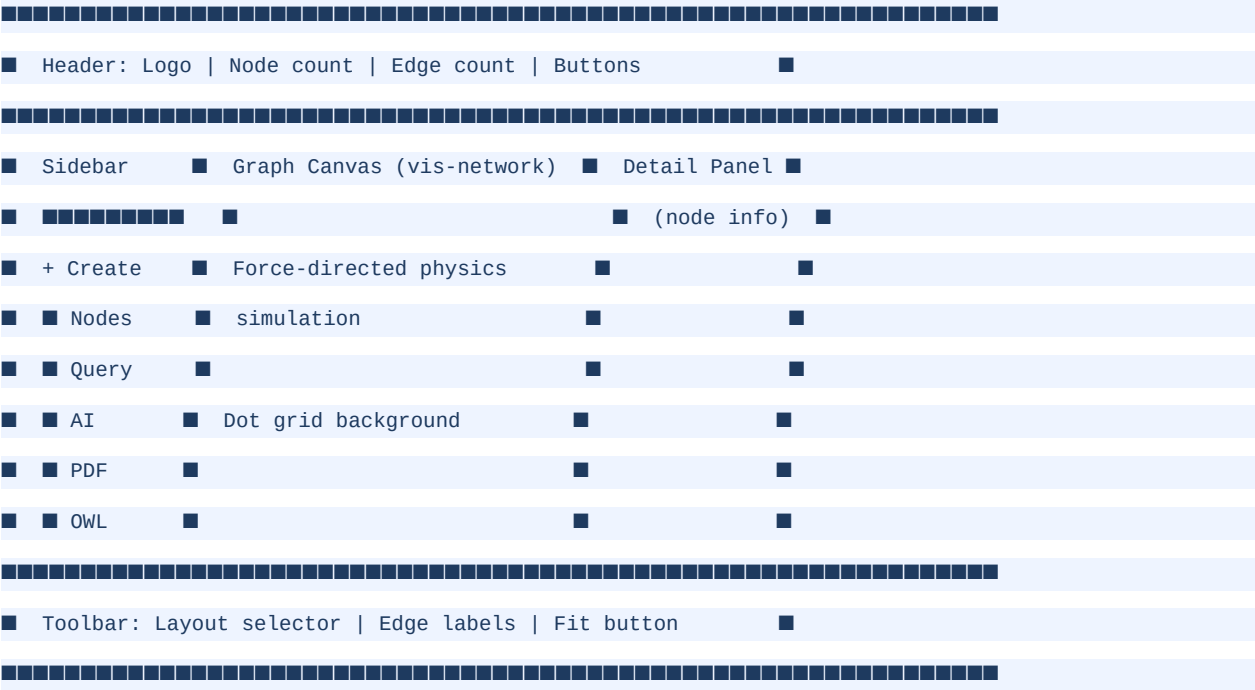
14. REST API Reference

Method	Endpoint	Request Body	Response
GET	/	—	Serves index.html frontend
GET	/graph	—	{nodes:[...], edges:[...]}
POST	/node	{label, type, properties}	{id, graph}
PUT	/node/<id>	{label, type, properties}	{graph}
DELETE	/node/<id>	—	{graph}
POST	/edge	{from, to, relation}	{id, graph}
DELETE	/edge/<id>	—	{graph}
POST	/upload	multipart/form-data PDF	{graph, stats, method}
POST	/query	{q: 'MATCH (n:Person)'}	{columns, rows, graph, message}
POST	/ask	{question: 'Who are...'}	{answer, relevant_node_ids, confidence}
POST	/owl/parse	{owl: '<rdf:RDF...>'}	{graph}
POST	/owl/import	{owl: '<rdf:RDF...>'}	{graph, added_nodes, added_edges}
POST	/clear	—	{graph: {nodes:[], edges:[]}}

15. Frontend Architecture

The entire frontend is a single file: templates/index.html (~1100 lines). It's a Single Page Application (SPA) with no build tools, no npm, no React — just HTML, CSS, and vanilla JavaScript.

Layout Structure



Key JavaScript Functions

Function	What it does
loadGraph()	Fetches /graph?t=timestamp, calls renderGraph()
renderGraph(data)	Destroys old vis-network, creates new one with nodes+edges
uploadPDF()	POSTs PDF to /upload, shows spinner, calls renderGraph on response
uploadMultiPDF()	Loops through files, calls /upload for each, shows progress bar
askAI()	POSTs question to /ask, highlights relevant_node_ids on graph
runQuery()	POSTs query to /query, renders result table + subgraph
showDetail(nodeId)	Opens right panel with node info, outgoing/incoming edges
toggleTheme()	Toggles :root.light class, saves to localStorage
clearAll()	POSTs to /clear, re-fetches graph to confirm empty

vis-network Configuration

The graph uses forceAtlas2Based physics solver with these settings:

```
physics: {
  solver: 'forceAtlas2Based',
  forceAtlas2Based: {
```

```
gravitationalConstant: -60, // repulsion between nodes
centralGravity: 0.005,      // pull toward center
springLength: 140,          // ideal edge length
springConstant: 0.08,       // edge stiffness
damping: 0.4                 // how fast motion stops
}
}
```

16. Interview Questions & Answers

Q1. What is this project?

A full-stack web app that reads research papers (PDFs), extracts entities and relationships using AI (Groq LLM), stores them in MongoDB Atlas, and visualizes them as an interactive knowledge graph.

Q2. What is a knowledge graph?

A graph where nodes are real-world entities (people, orgs, concepts) and edges are relationships. Example: 'BERT' -[DEVELOPED_BY]→ 'Google' -[LOCATED_IN]→ 'USA'.

Q3. Why Python 3.11 specifically?

Python 3.14 has a TLS incompatibility with MongoDB Atlas — the SSL handshake fails with TLSV1_ALERT_INTERNAL_ERROR. Python 3.11 works correctly with MongoDB's TLS configuration.

Q4. How does PDF extraction work?

pdfplumber extracts text → split into 3000-char chunks → each chunk sent to Groq LLM → LLM returns JSON {entities, relations} → deduplicated → saved to MongoDB.

Q5. What happens if Groq API is down?

The code has a fallback chain. If Groq fails, it automatically uses spaCy NER + SVO dependency parsing. If all chunks fail, spaCy is used. The user always gets results.

Q6. How do you prevent duplicate nodes?

MongoDB upsert=True with unique index on id field. Same ID never creates a duplicate — it just updates the existing document.

Q7. How does the NL query work?

Load graph from MongoDB → build compact text summary (max 150 nodes) → send question + summary to Groq LLM → LLM returns answer + relevant node IDs → frontend highlights those nodes.

Q8. What is the database schema?

Two MongoDB collections: nodes {id, label, type, properties} and edges {id, from_id, to_id, label, properties}. Indexes on id (unique) and compound (from_id, to_id, label).

Q9. How does BFS shortest path work?

Build adjacency list from all edges (bidirectional). BFS queue carries (current_node, path_nodes, path_edges). When destination reached, return the path. visited set prevents infinite loops.

Q10. How would you scale this for production?

Replace tlsInsecure=True with proper TLS certs. Add JWT authentication. Use Gunicorn with multiple workers. Add Redis caching. Use Celery for background PDF processing. Add rate limiting on AI endpoints.

Q11. What is tlsInsecure=True and why is it used?

It bypasses TLS certificate validation in PyMongo. Used because Python 3.11's OpenSSL sends a TLS handshake that MongoDB Atlas rejects with strict cert validation. Acceptable for development, not production.

Q12. How does the database abstraction work?

database.py uses module-level globals (_db, _use_mongo). Every function checks _is_mongo() and branches to MongoDB or SQLite code. app.py calls insert_node() without knowing which DB is active.